

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..16.7

TABLES

 Copy as Markdown

Tables

Storing a [data model](#) in the database takes one line: add the `table` key to its model file, and the class gains a generated `db` field with typed methods for reading and writing rows. From there the path is short: create and apply a [migration](#) (press **M** in the `serverpod start` terminal), then call the [CRUD methods](#) from your endpoints. The rest of this page covers what the table definition itself can do.

```
class: Company
table: company
fields:
  name: String
```

When the `table` keyword is added to the model, Serverpod generates new methods for [interacting](#) with the database. The keyword is also picked up when a migration is created, which generates the [migrations](#) needed to update the database.

For the full list of keywords you can use in a model file, see the [Model reference](#). For how each type is stored in Postgres, SQLite, and JSON, see [Field types](#).

INFO

When you add a `table` to a serializable class, Serverpod will automatically add an `id` field of type `int?` to the class. You should not define this field yourself. The `id` is set when you interact with an object stored in the database.

Choosing where a table lives

By default, a table model generates its table on the server, and the client package gets the same class model without database bindings. The `database` keyword changes this. It accepts four values:



Ask AI

Value	Description
<code>server</code>	Generates tables only on the server, and a non-table model on the client package (default).
<code>client</code>	Generates tables only on the client, and a non-table model on the server package.
<code>all</code>	Generates table models on both server and client.
<code>sync</code>	Experimental. Like <code>all</code> , and additionally keeps the table synchronized between the client and the server. See Sync tables with the server .

For example, to store the table only on the device:

```
class: Company
table: company
database: client
```

A client table lives in a SQLite database on the device, which is useful for local caches and offline data. It gets its own SQLite [migrations](#) in the client package. With `all`, the server and client tables are separate databases, and data does not move between them automatically.

For how to use the database on the device, see [Client-side database](#).

Non persistent fields

You can opt out of creating a column in the database for a specific field by using the `!persist` keyword.

```
class: Company
table: company
fields:
  name: String, !persist
```

All fields are persisted by default and have an implicit `persist` set on each field.

Data representation

A field with a primitive / core Dart type is stored as its respective column type. An `int` field is Postgres `bigint` (64-bit), not `integer`. Complex types, such as another model, a `List`, or a `Map`, are stored as a

`json` column in the database by default. Fields of type `dynamic` are stored the same way, and can always hold null. See [Field types](#) for the full YAML-to-column mapping, including SQLite.

```
class: Company
table: company
fields:
  address: Address # Stored as a json column
```

This means that each row has its own copy of the nested object that needs to be updated individually. If you instead want to reference the same object from multiple different tables, you can use the `relation` keyword.

This creates a database relation between two tables and always keeps the data in sync.

```
class: Company
table: company
fields:
  address: Address?, relation
```

For a complete guide on how to work with relations, see the [Relations](#) section.

Storing serializable fields as JSONB

By default, complex types are stored as `json` in the database. You can opt into `jsonb` storage instead using the `serializationDataType` keyword. JSONB is a binary format that supports efficient querying and [GIN indexing](#) for PostgreSQL.

INFO

The `serializationDataType` keyword is only valid on serializable field types (nested models, Lists, Maps, Sets, records, custom classes, and `dynamic`). Primitive types like `String` and `int` have their own native database column types and are not affected by this setting.

You can set `serializationDataType` at three levels, each overriding the one above it:

Field level

Applies to a single field:

```
class: Product
table: product
fields:
  tags: List<String>, serializationDataType=jsonb
  metadata: Map<String, String>, serializationDataType=jsonb
```

Class level

Applies to all serializable fields in the class. Can be overridden by field-level setting.

```
class: Product
table: product
serializationDataType: jsonb
fields:
  tags: List<String> # Stored as jsonb
  metadata: Map<String, String> # Stored as jsonb
  name: String
  history: List<String>, serializationDataType=json # Stored as json (override)
```

Project level

Applies to all models in the project. Add this to your `config/generator.yaml`:

```
serialize_as_jsonb_by_default: true
```

When enabled, all serializable fields across all models default to `jsonb` unless overridden at the class or field level.

Migrating between json and jsonb

If you change the `serializationDataType` between `json` and `jsonb` at any level, the migration system will convert existing columns automatically with no data loss.

Choosing an ID strategy

Serverpod supports two id types: `int` (the default) and `UuidValue`. The right choice depends on how the id is generated and where it is exposed.

An `int` id defaults to `serial`, which means the database assigns an auto-incrementing integer when the row is inserted. Serial ids are compact and sequential, which keeps indexes efficient and makes them a good fit for internal references. The trade-offs are that the id only exists after the row is saved, and the values are guessable and enumerable, so a sequential id in a public URL can leak how many records you have or let someone probe for others.

A `UuidValue` id is a 128-bit random value. It is not enumerable, and it can be generated before the row is inserted, which suits ids that appear in public URLs, are created offline on the client, or are merged from several sources. UUIDs are larger than integers and random rather than sequential, so index locality is slightly worse than with serial ids.

As a rule of thumb, use the default `int` serial id for internal tables, and reach for `UuidValue` when the id is public-facing, needs to exist before the row is stored, or is created across more than one system. Because a UUID can be generated on the client, never treat a client-supplied id as proof that the caller owns a record; always check that the authenticated user is allowed to use it.

Change ID type

Changing the type of the `id` field allows you to customize the identifier type for your database tables. This is done by declaring the `id` field on table models with one of the supported types. If the field is omitted, the id field will still be created with type `int` by default.

The following types are supported for the `id` field:

Type	Default	Default Persist options	Default Model options	Description
<code>int</code>	serial	serial (optional)	-	64-bit serial integer (<code>bigint</code> / <code>bigserial</code>).
<code>UuidValue</code>	random	random, random_v7	random, random_v7	UUID v4 (<code>random</code>) or v7 (<code>random_v7</code>).

Declaring a custom ID type

To declare a custom type for the `id` field in a table model file, use the following syntax:

```
class: UuidIdTable
table: uuid_id_table
fields:
  id: UuidValue?, defaultPersist=random
```

```
class: IntIdTable
table: int_id_table
fields:
  id: int?, defaultPersist=serial # The default keyword for 'int' is optional.
```

Default Uuid model value

For UUIDs, it is possible to configure the `defaultModel` value. This will ensure that UUIDs are generated as soon as the object is created, rather than when it is persisted to the database. This is useful for creating objects offline or using them before they are sent to the server.

```
class: UuidIdTable
table: uuid_id_table
fields:
  id: UuidValue, defaultModel=random
```

When using `defaultModel=random` or `defaultModel=random_v7`, the UUID will be generated when the object is created. Since an id is always assigned the `id` field can be non-nullable.

To have the database assign the id on insert instead of generating it when the object is created, use `defaultPersist=random` or `defaultPersist=random_v7` without `defaultModel`.

Column name override

By default, the column name in the database is the same as the name of the field in the model. However, you can override this to use a different name by using the `column` keyword. For example, if you have a model property named `userName` you might want to store it in the database as `user_name`.

To override the column name, use the `column` keyword on the field definition:

```
class: User
table: users
fields:
  userName: String, column=user_name
```

INFO

When adding a column name override to an existing model, the next migration will contain a column rename operation to rename the column in the database.

Restrictions:

- The `id` field cannot have a column name override.
- The column name must be unique within the model.
- On a relation, set `column` on the [foreign key field](#), not on the object relation field.

Relations

You can use the `column` keyword to override the name of the foreign key field in a relation.

```
# Employee
class: Employee
table: employee
fields:
  name: String
```

```

    departmentId: int, relation(name=department_employees, parent=department),
    column=fk_employee_department_id

# Department
class: Department
table: department
fields:
    name: String
    employees: List<Employee>?, relation(name=department_employees)

```

Overriding the column name is also supported when the field is used in an index. Note that the index uses the name of the field, not the column name.

```

# Contractor
class: Contractor
table: contractor
fields:
    name: String
    serviceIdField: int?, column=fk_contractor_service_id
    service: Service?, relation(field=serviceIdField)
indexes:
    contractor_service_unique_idx:
        fields: serviceIdField
        unique: true

# Service
class: Service
table: service
fields:
    name: String
    description: String?

```

Inheritance

Besides column override being a database-level feature, it is allowed to be set on non-table models to support [inheritance](#). If a base class has a column name override, all child classes that implement a table will inherit the column name override. This is especially useful when you have multiple database tables that share similar column names.

```

# Entity
class: Entity
fields:
    name: String
    createdAt: DateTime, default=now, column=created_at
    updatedAt: DateTime, default=now, column=updated_at

# Employee
class: Employee
extends: Entity
table: employee
fields:

```

```
    departmentId: int, relation(name=department_employees, parent=department),  
    column=fk_employee_department_id
```

```
# Department
```

```
class: Department
```

```
extends: Entity
```

```
table: department
```

```
fields:
```

```
    employees: List<Employee>?, relation(name=department_employees)
```

In the above example, the `Employee` and `Department` classes will receive the `createdAt` and `updatedAt` fields from the `Entity` class with the columns overridden to `created_at` and `updated_at`, respectively.